K NEAREST NEIGHBORS IMPLEMENTATION IN MAPREDUCE

Database Management

March 2026

Kenza Chaabouni[1]
Hitaishi Dhoowooah[2]

[1]kenza.chaabouni@polytechnique.edu
[2]hitaishi.dhoowooah@polytechnique.edu

1 INTRODUCTION

The K-Nearest Neighbor (KNN) algorithm is one of the most well-known and widely used algorithms in data mining and machine learning [1]. It is very simple and effective, making it a popular choice for classification tasks in a wide range of domains, from biomedicine to social media analysis. The KNN algorithm used for classification works by finding the k closest training examples to an unseen test point and assigning the majority class among the neighbors as the predicted label. Despite being conceptually simple, KNN is computationally expensive. This is because for each test point, distances must be computed against every training example. This results in a complexity of O(nd) where n is the number of training instances and d is the number of features. Therefore, KNN becomes impractical for large datasets where both runtime and memory consumption become prohibitive.

The rise of big data makes this limitation increasingly relevant. Datasets with millions of training samples are very common and traditional implementations of KNN on a single node are no longer feasible. This motivates the development of distributed approaches that leverage parallel computing frameworks to make KNN scalable to arbitrarily large datasets.

MapReduce is a commonly used parallel programming paradigm to process large datasets across a cluster of computing nodes [2]. By dividing computations into a map phase and a reduce phase, scalable data processing can be achieved. Apache Spark makes use of this paradigm with in-memory computation capabilities, making it suitable for iterative data mining and machine learning tasks.

In this project, a MapReduce-based KNN algorithm, proposed by Maillo et al. (2015) [3], is studied and implemented. The KNN classification task is parallelized by splitting the training set, computing local k-nearest neighbors in each chunk in the map phase, and merging the results in the reduce phase to obtain the exact global k-nearest neighbors.

The implementation is done in PySpark in both RDD and DataFrame, using the Poker Hand dataset from the UCI Machine Learning Repository [2], as used in the paper. An experimental analysis is carried out by varying parameters such as the dataset size, the number of mappers, and k. The performance of both approaches is then compared and analyzed. The report proceeds as follows: Section 2 presents the methodology; Section 3 describes the implementation; Section 4 details the experimental analysis; Section 5 discusses the results, and Section 5 concludes.

2 METHODOLOGY

The KNN implementation in MapReduce is composed of a map phase, a reduce phase, and majority voting. First, the training dataset is partitioned across map tasks which compute local nearest neighbors. Then, the results get sent to a single reducer which merges them to determine the global k nearest neighbors for each test point. We detail this methodology below and provide the algorithms used for each step [3].

[3] We note that all algorithms shown in this report are taken from the Maillo et al. (2015) paper [3].

2.1 Map phase

Each map task Map_j operates on a disjoint subset TR_j of the training set, formed by assigning consecutive HDFS blocks to each mapper. This ensures that there is no overlap between tasks. The test dataset TS is however not partitioned in order to to make sure that each map task computes the distances with all test points.

Algorithm 1 Map function

Require: TS, k
Constitute TR_j with the instances of split j
for $i = 0$ to $i < \text{size}(TS)$ **do**
  Compute k-NN $(x_{\text{test},i}, TR_j, k)$
  for $n = 0$ to $n < k$ **do**
    $CD_j(i, n) = \langle \text{Class}(\text{neigh}_n), \text{Dist}(\text{neigh}_n) \rangle_i$
  end for
end for
$key \leftarrow idMapper$
EMIT $\langle key, CD_j \rangle$

For each test instance $x_{\text{test},i}$, the mapper computes its k nearest neighbors within TR_j and stores the result as a row in a local matrix CD_j, where each entry holds a $\langle \text{class}, \text{distance} \rangle$ pair, sorted in ascending order by distance.

Once all test instances have been processed, the mapper emits a single key-value pair $\langle idMapper, CD_j \rangle$, where the key identifies the mapper and the value is the full $n \times k$ candidate matrix.

The test file is accessed line by line to avoid loading the entire set into memory simultaneously. The detailed algorithm is shown in Algorithm 1 above.

- **Complexity**

Each mapper computes pairwise distances between all n_{test} and its n_{train}/m training instances. This has a complexity of $O\left(n_{\text{test}} \times \frac{n_{\text{train}}}{m} \times d\right)$ for each mapper. Therefore, the per-mapper complexity scales as $\frac{1}{m}$.

2.2 Reduce Phase

The reduce phase is handled by a single reducer, which receives the candidate matrices CD_j from all mappers and progressively refines them into a global set of k nearest neighbors for each test instance.

Algorithm 2 Reduce operation

Require: $\text{size}(TS)$, k, CD_j
Require: Setup procedure has been launched
for $i = 0$ to $i < \text{size}(TS)$ **do**
  $cont \leftarrow 0$
  for $n = 0$ to $n < k$ **do**
    if $CD_j(i, cont).\text{Dist} < CD_{\text{reducer}}(i, n).\text{Dist}$ **then**
      $CD_{\text{reducer}}(i, n) \leftarrow CD_j(i, cont)$
      $cont \leftarrow cont + 1$
    end if
  end for
end for

Before receiving the output of the map, the reducer assigns a global matrix CD_{reducer} of size $n \times k$, with arbitrary class labels and distances set to $+\infty$. This is performed only once.

As each mapper's CD_j arrives, the reducer merges it into CD_{reducer} using an $O(k)$ merge of two sorted lists. For each test instance i, the distances in CD_j and CD_{reducer} are compared one by one starting from the closest neighbor, and CD_{reducer} is updated wherever a smaller distance is found. This iterative process is repeated for every incoming CD_j, so that after all mappers have been processed, CD_{reducer} contains the definitive global k nearest neighbors for every test instance. The detailed algorithm is shown in Algorithm 2 above.

Algorithm 3 Reduce cleanup process

Require: size(TS), k
Require: Reduce operation has finished
1: **for** $i = 0$ to $i <$ size(TS) **do**
2: $PredClass_i \leftarrow$ MajorityVoting(Classes(CD_{reduce}))
3: $key \leftarrow i$
4: **EMIT** $\langle key, PredClass_i \rangle$
5: **end for**

Once all map outputs have been merged, the cleanup phase performs the final classification. For each test instance $x_{\text{test},i}$, a majority vote is applied over the k class labels stored in $CD_{\text{reducer}}[i]$, and the most frequent class is assigned as the predicted label. The result is emitted as $\langle i, \hat{y}_i \rangle$, where i is the index of the test instance and $\hat{y}_i$ is its predicted class. This produces a single output file containing predictions for the entire test set. The detailed algorithm is shown in Algorithm 3 above.

- Complexity

For each incoming CD_j, the reducer performs an $O(k)$ merge of two sorted lists per test instance. This is repeated for all m mappers and all n_{test}. It follows that the total complexity of the reduce operation is:

$$O(m \times n_{\text{test}} \times k)$$

In addition, the majority vote over k neighbors for each of the n_{test} costs $O(k)$. This results in a total of $O(n_{\text{test}} \times k)$ which is much smaller than the complexity of the map and reduce algorithms.

We observe that the map phase dominates complexity-wise. However, its cost scales as $1/m$, it follows that adding mappers reduces the map phase's complexity linearly.

The total complexity [4] of the MapReduce KNN is $O\left(\frac{n_{\text{test}} \times n_{\text{train}} \times d}{m}\right)$. This represents a linear speedup of m over the sequential approach's complexity which is $O(n_{\text{test}} \times n_{\text{train}} \times d)$.

2.3 Dataset

We test this approach's performance on the PokerHand dataset from the the UCI repository [2], containing 1,025,010 data points. Each instance represents a 5-card poker hand, described by 10 features (suit and rank of every card) and with one of 10 possible hand ranking (e.g., full house, royal flush ,etc). The original dataset contains 25,010 training and 1,000,000 test instances. In our implementation, we only use 5,000 points of the test dataset. In addition, we start by using 10% of the training dataset for our initial analysis. We further vary the fraction of the training dataset progressively, from 10% to 100% across five steps: 0.1, 0.25, 0.5, 0.75, 1.0, to evaluate scalability.

[4] We note that the reduce complexity is omitted from the total complexity as it is negligible relative to the map complexity.

3 IMPLEMENTATION

We implement the methodology explained above using both an RDD and a DataFrame approaches.

3.1 RDD Implementation

The RDD implementation uses `mapPartitionsWithIndex` for the map phase, where each partition corresponds to a mapper Map_j. The full test set is broadcast to all executors via `sc.broadcast`, so each mapper has access to all test instances without shuffling. Distances are computed using a loop over all test-training pairs. In the reducer, we use `aggregateByKey` to iteratively merge incoming CD_j matrices into CD_{reducer}. The majority vote uses `flatMap`.

3.2 DataFrame Implementation

The DataFrame implementation uses `applyInPandas` on a grouped DataFrame. This enables fully vectorized distance computation using numpy matrix operations: $|a-b|^2 = |a|^2 + |b|^2 - 2(a \cdot b)$, computing all pairwise distances in a single matrix operation. This explains why the test set is pre-materialized as a numpy matrix on the driver and passed directly to each partition function. The reduce phase is then performed on the driver after collecting all map outputs, iterating over partition boundaries to merge CD_j matrices sequentially using the same $O(k)$ two-pointer merge.

4 EXPERIMENTAL ANALYSIS

The experimental analysis was performed in two parts. In Experiment 1, we implement 5-fold cross-validation, as described in the paper. We combine 10% of training data with 5000 test points to give a dataset of around 7500 points, which we use for cross-validation. We run both sequential KNN and MapReduce-based KNN for each fold with varying number of neighbors k and number of mappers m. We obtain as results the: average accuracy, average map time, average reduce time and average global time. These results are used to calculate the speedup obtained from the MapReduce approach over the sequential approach.

In Experiment 2, we focus on the performance of the MapReduce implementation. We perform two experiments. In Experiment 2.1, we varied the size of the training set by choosing a fraction of the training dataset, and the number of neighbors k, while keeping the number of maps, m, constant at 16. In Experiment 2.2, we varied the number of maps and the number of neighbors k, while using the full training dataset. The parameters used are shown in Table 1.

Table 1: Parameters for Experiment 2

Parameter	Experiment 2.1 values	Experiment 2.2 values
fraction	[0.1, 0.25, 0.5, 0.75, 1.0]	1.0
k	[1, 3, 5, 7]	[1, 3, 5, 7]
m	16	[16, 32, 64, 128]

4.1 Speedup

The speedup is calculated as follows:

$$Speedup = \frac{reference \quad time}{parallel \quad time}$$

The reference time is the runtime corresponding to the sequential KNN algorithm whereas the parallel is the runtime corresponding to the MR-KNN algorithm. The speedup results are obtained from Experiment 1. As shown in Table 2, we see speedup values of ~6x for MR-KNN over sequential KNN. This validates the use of the MapReduce approach for faster inference. Furthermore, increasing the number of map tasks should theoretically increase speedup values, as presented in the reference paper. However, we observe lower speedup values for increasing number of map tasks. This occurs because our experiments were run on a single-node PySpark environment. While the implementation correctly follows the MapReduce paradigm and would scale linearly on a multi-node cluster, the observed speedups are limited by the single-node setup [5] . On a true distributed cluster as used in the paper, each partition would execute on a separate physical node simultaneously, yielding the near-linear speedups.

The DataFrame implementation shows speedup values consistently below 1.0 across all configurations, meaning it is always slower than its sequential baseline. This is explained by the overhead introduced by `applyInPandas`, which requires serialising each training partition from the JVM into a pandas DataFrame in Python, executing the UDF, and deserialising the results back. This serialisation cost is paid for every partition on every run, and on a single node it far outweighs any computational savings from partitioning. Additionally, the use of `json.dumps` and `json.loads` to pass the CD_j matrix through the DataFrame schema introduces further overhead that does not exist in the RDD implementation. In contrast, the sequential baseline for the DataFrame implementation benefits from fully vectorised numpy operations without any Spark overhead, making it disproportionately fast relative to the parallelised version on a single node. On a true multi-node cluster, the serialisation cost would be amortised across many machines and the DataFrame implementation would likely achieve competitive performance.

4.2 Accuracy

The performance of the model in terms of accuracy is measured mainly in Experiment 2.1. It is observed that the accuracy generally increases as the size of the training set increases, as well as, as k increases. This is expected since a larger training set provides a more representative sample of the underlying data distribution. This means that the KNN classifier more neighbors to choose from , thus reducing the likelihood of misclassification due to sparse coverage of the feature space. Similarly, increasing k reduces the sensitivity of the classifier to noise and outliers. With a larger neighborhood, the majority vote becomes more robust as individual noisy points have less influence on the final prediction.

[5] We note that all experiments were run on a Linux machine equipped with an Intel Xeon W-1290P CPU (10 physical cores, 20 threads, up to 5.3 GHz) and 62 GB of RAM, using Python 3.12.7. Runtime measurements correspond to wall-clock time.

Table 2: Experiment 1: Speedup Results

k	# maps	RDD Speedup	DF Speedup
1	sequential	1.0000	1.0000
	16	5.9336	0.1530
	32	5.5226	0.1012
	64	4.5074	0.0636
	128	3.3328	0.0362
	256	2.1320	0.0198
3	sequential	1.0000	1.0000
	16	5.7650	0.1280
	32	5.3912	0.0824
	64	4.3076	0.0498
	128	3.1262	0.0286
	256	1.9918	0.0150
5	sequential	1.0000	1.0000
	16	5.9696	0.1224
	32	5.1852	0.0732
	64	4.2408	0.0434
	128	3.0524	0.0240
	256	1.9202	0.0120
7	sequential	1.0000	
	16	5.9802	0.1104
	32	5.0686	0.0644
	64	4.0936	0.0378
	128	2.9140	0.0206
	256	1.8126	0.0106

These findings are consistent with the results reported in the reference paper, where higher values of k yielded better accuracy on the Poker Hand dataset for both the sequential and parallel versions of the algorithm.

The results from Experiment 2.2 also provide insights about accuracy. In fact, the MR-KNN algorithm gives roughly the same classification accuracy regardless of the number of map tasks used. This is a key property of the proposed approach since it was based on the principle that partitioning the training data across map tasks does not introduce any approximation because the reduce phase recovers the globally closest k neighbors by merging all local results.

As shown in Table 3, both implementations achieve comparable accuracy values across all configurations. The small differences observed between the RDD and DataFrame implementations are attributable to differences in how the two approaches partition the training data. Since partitioning is not deterministic across implementations, slightly different training instances may be assigned to each partition, leading to minor variations in the local k-nearest neighbor lists. However, these differences are negligible and do not affect the overall conclusions about the algorithm's accuracy.

4.3 Scalability

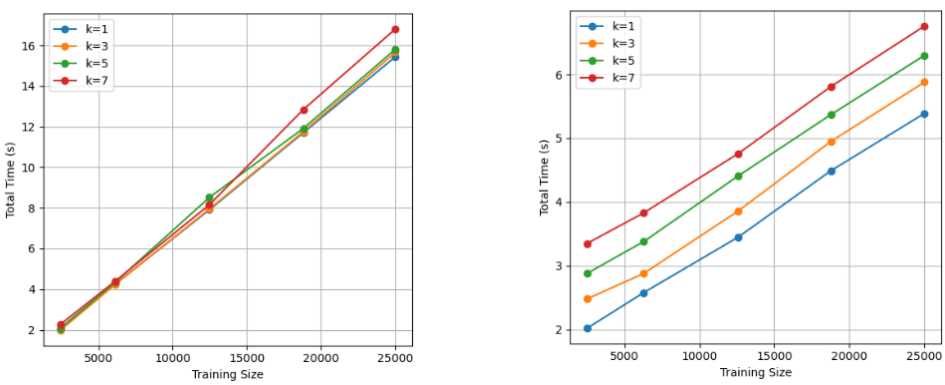
The scalability of the MR-KNN algorithm is assessed by comparing the total time of execution of the map and reduce phases with increasing training set size. This uses results from Experiment 2.1, where the train set fraction was varied. It is observed that as the training set size increases, the total time increases, as shown in

Table 3: Accuracy Results from Experiment 2.1

k	Train set fraction	RDD Accuracy	DF Accuracy
1	0.1	0.4830	0.4760
	0.25	0.4942	0.4900
	0.5	0.5068	0.5010
	0.75	0.5004	0.5018
	1.0	0.5060	0.5034
3	0.1	0.5066	0.5054
	0.25	0.5226	0.5134
	0.5	0.5318	0.5212
	0.75	0.5360	0.5240
	1.0	0.5276	0.5374
5	0.1	0.5314	0.5206
	0.25	0.5386	0.5336
	0.5	0.5508	0.5342
	0.75	0.5468	0.5398
	1.0	0.5538	0.5514
7	0.1	0.5286	0.5298
	0.25	0.5460	0.5402
	0.5	0.5576	0.5456
	0.75	0.5570	0.5494
	1.0	0.5676	0.5648

Table 4 and Figure 1. This is expected since a larger training set means each map task must compute distances between all test points and a larger number of training instances, directly increasing the computational cost of the map phase.

The total time increases slightly with k. This is because larger values of k require maintaining and merging larger candidate lists in both the map and reduce phases. Each map task must track k neighbors per test point instead of fewer, and the reduce phase must merge larger CD_j matrices. However, as noted in the paper, this effect is minor due to the efficient encoding of the neighbor lists.



(a) RDD (b) DataFrame

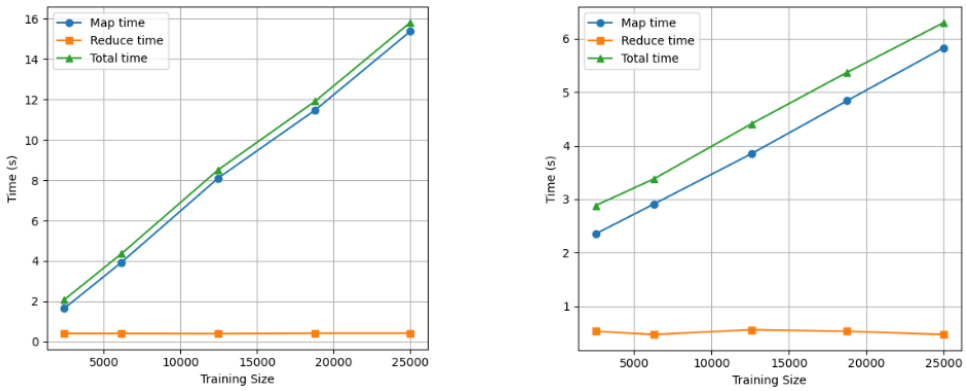
Figure 1: Total execution time vs training set size for varying k (Experiment 2.1, m=16)

In terms of scalability, the relationship between training set size and total execution time appears approx-

Table 4: Scalability Results from Experiment 2.1

k	Train set fraction	RDD Total Time (s)	DF Total Time (s)
1	0.1	2.07	2.02
	0.25	4.27	2.58
	0.5	7.91	3.45
	0.75	11.71	4.49
	1.0	15.45	5.39
3	0.1	1.97	2.48
	0.25	4.25	2.88
	0.5	7.95	3.86
	0.75	11.76	4.95
	1.0	15.67	5.88
5	0.1	2.08	2.88
	0.25	4.36	3.38
	0.5	8.52	4.41
	0.75	11.92	5.37
	1.0	15.82	6.30
7	0.1	2.26	3.35
	0.25	4.41	3.83
	0.5	8.16	4.76
	0.75	12.85	5.81
	1.0	16.82	6.76

imately linear, which is consistent with the theoretical complexity of the algorithm. Since each map task independently processes its local partition of the training data against the full test set, the workload per task grows linearly with the number of training instances assigned to it. This confirms that the MR-kNN algorithm scales predictably with dataset size, and that adding more training data does not introduce any superlinear overhead in the map phase.



(a) RDD (b) DataFrame

Figure 2: Map vs Reduce time breakdown for k=5 (Experiment 2.1, m=16)

Furthermore, the total time can be split into Map time and Reduce time to obtain a more fine-grained analysis. It is observed that the map phase dominates the total execution time across all configurations, while the reduce phase remains negligible by comparison. This is consistent with the complexity analysis shown in Section 2. The map vs reduce time breakdown for k=5 is illustrated in Figure 2, for both RDD and DataFrame.

In fact, the map time scales with increasing training set size while the reduce time remains almost constant. This is consistent with the findings of the reference paper, which observed that the reduce phase accounts for only a small fraction of the total runtime regardless of the number of mappers or the value of k.

However, it should be noted that the scalability benefits of the MR-kNN algorithm in terms of parallel speedup could not be fully demonstrated in this experimental setting. As discussed, the single-node environment used for these experiments does not support true distributed parallelism, which limits the conclusions that can be drawn about the algorithm's behavior on a real multi-node cluster.

4.4 RDD vs DataFrame Comparison

Table 5: RDD vs DataFrame: Performance Comparison (Experiment 2.1, m=16)

k	Fraction	RDD Time (s)	DF Time (s)	RDD Accuracy	DF Accuracy
1	0.10	2.07	2.02	0.4830	0.4760
	0.25	4.27	2.58	0.4942	0.4900
	0.50	7.91	3.45	0.5068	0.5010
	0.75	11.71	4.49	0.5004	0.5018
	1.00	15.45	5.39	0.5060	0.5034
3	0.10	1.97	2.48	0.5066	0.5054
	0.25	4.25	2.88	0.5226	0.5134
	0.50	7.95	3.86	0.5318	0.5212
	0.75	11.76	4.95	0.5360	0.5240
	1.00	15.67	5.88	0.5276	0.5374
5	0.10	2.08	2.88	0.5314	0.5206
	0.25	4.36	3.38	0.5386	0.5336
	0.50	8.52	4.41	0.5508	0.5342
	0.75	11.92	5.37	0.5468	0.5398
	1.00	15.82	6.30	0.5538	0.5514
7	0.10	2.26	3.35	0.5286	0.5298
	0.25	4.41	3.83	0.5460	0.5402
	0.50	8.16	4.76	0.5576	0.5456
	0.75	12.85	5.81	0.5570	0.5494
	1.00	16.82	6.76	0.5676	0.5648

Table 5 presents a side-by-side comparison of the RDD and DataFrame implementations in terms of execution time and accuracy across all experimental configurations. Both implementations achieve comparable accuracy values, which is expected since both implement the same MR-KNN algorithm and the reduce phase guarantees exact recovery of the global k-nearest neighbors regardless of the implementation approach.

In terms of execution time, the DataFrame implementation is consistently faster than the RDD implementation at larger training set sizes. This is primarily because of the fully vectorised numpy distance computation used in the DataFrame implementation. Rather than iterating over training instances one by one as in the RDD approach, the DataFrame implementation computes the entire $n_{test} \times n_{train_partition}$ distance matrix in a single matrix operation using the identity $||a-b||^2 = ||a||^2 + ||b||^2 - 2(a \cdot b)$. This approach leverages highly optimised linear algebra, resulting in significantly faster distance computations particularly as the training set grows larger.

At smaller training set sizes (fraction = 0.1), the two implementations perform similarly, and in some cases the RDD implementation is marginally faster. This is because at small scales the numpy vectorisation advantage

is less pronounced, while the overhead of `applyInPandas` serialisation in the DataFrame implementation is relatively more significant. As the training set grows, the vectorisation advantage dominates and the DataFrame implementation pulls ahead. This crossover behaviour confirms that the DataFrame implementation is better suited for larger datasets, while the RDD implementation incurs lower fixed overhead at smaller scales.

Table 6: RDD vs DataFrame: Performance Comparison (Experiment 2.2, fraction=1.0)

k	# Maps	RDD Time (s)	DF Time (s)	RDD Accuracy	DF Accuracy
1	16	16.44	5.38	0.5060	0.5034
	32	15.94	6.64	0.5072	0.5066
	64	16.24	9.24	0.5036	0.5110
	128	16.34	15.29	0.5054	0.5104
3	16	16.21	5.88	0.5276	0.5374
	32	15.89	7.59	0.5342	0.5276
	64	16.55	11.26	0.5330	0.5346
	128	16.95	18.94	0.5326	0.5330
5	16	16.70	6.31	0.5538	0.5514
	32	16.19	8.66	0.5572	0.5536
	64	16.86	13.30	0.5502	0.5554
	128	17.50	23.10	0.5540	0.5524
7	16	16.70	6.77	0.5676	0.5648
	32	16.38	9.45	0.5670	0.5658
	64	17.10	15.27	0.5698	0.5714
	128	18.27	26.83	0.5678	0.5648

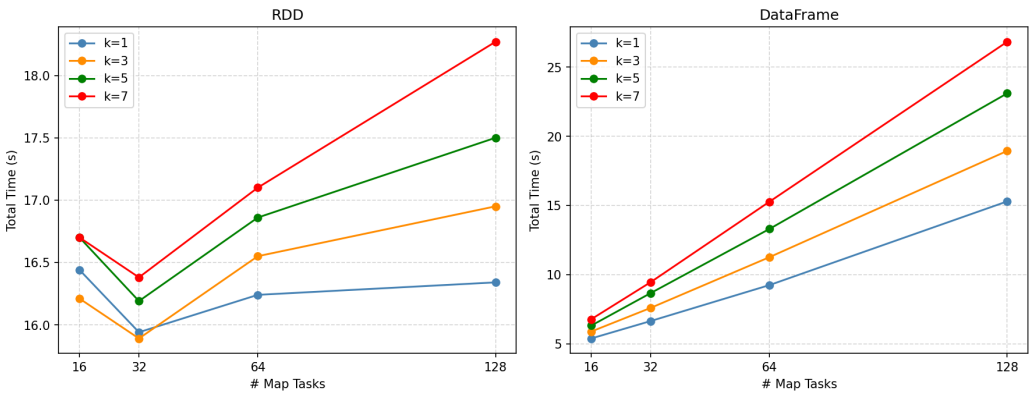


Figure 3: Total execution time vs number of map tasks (experiment 2.2, fraction=1.0)

In Experiment 2.2, a different pattern emerges between the two implementations. The RDD implementation shows remarkably stable total execution times across all mapper configurations, ranging from approximately 15 to 18 seconds regardless of the number of map tasks or the value of k. This stability arises because the RDD approach operates natively within the JVM and Spark's task scheduler is able to efficiently pipeline map tasks on a single node. Thus, the overhead of additional partitions is absorbed without significantly increasing total runtime. The slight increase in reduce time with more mappers is consistent with the paper's observation that more mappers produce more CD_j matrices to merge, but this effect remains small.

The DataFrame implementation shows the opposite trend. Total execution time increases consistently with the number of map tasks, from approximately 5–7 seconds at m=16 up to 18–27 seconds at m=128. This is explained by the applyInPandas serialisation overhead, as explained before. Furthermore, with more partitions each partition contains fewer training instances, which reduces the size of the distance matrix per partition and diminishes the numpy vectorisation advantage that makes the DataFrame implementation fast at lower map counts.

A key crossover point is visible in Table 6. At m=16, the DataFrame implementation is consistently faster than RDD across all k values, with execution times approximately 3x lower. As the number of mappers increases, the two implementations converge, at approximately m=128 for most k values. Beyond this point, the DataFrame implementation becomes slower than the RDD implementation. This crossover behaviour confirms that the DataFrame implementation is best suited for low mapper counts on single-node environments, while the RDD implementation is more robust to increasing parallelism due to its lower per-partition overhead.

Experiment 2.2 confirms that accuracy values remain consistent between the two implementations and are not affected by the number of map tasks.

5 RESULTS AND DISCUSSION

5.1 Obtained results

5.1.1 • Strong Points of MR-KNN

The MR-KNN algorithm presents several strong points that make it a compelling approach for big data classification. First, the algorithm provides **exact classification accuracy**. It is not an approximation of KNN but a faithful parallelisation that always produces the same predictions as the standard sequential algorithm. This is guaranteed by the reduce phase, which recovers the globally closest k neighbors by merging all local candidate lists, regardless of how the training data is partitioned.

The algorithm exhibits **linear scalability** with respect to training set size, as demonstrated in Experiment 2.1. Since each map task independently processes its local partition without any data exchange with other partitions, the computational cost grows proportionally to the number of training instances. This makes the algorithm well-suited for datasets of arbitrary size, provided that sufficient computing resources are available.

The **reduce phase is negligible** in terms of computational cost compared to the map phase, as consistently observed across all experimental configurations in both Experiment 2.1 and Experiment 2.2. This is an important property since it means the bottleneck of the algorithm lies entirely in the map phase, which is fully parallelizable across nodes. Adding more computing nodes therefore directly reduces total runtime without introducing significant reduce-phase overhead.

Finally, the algorithm is **fault tolerant** from the use of the MapReduce paradigm. If a map task fails, it can be restarted on another node without affecting the correctness of the result, making the approach robust for long-running computations on large datasets.

5.1.2 • Weak Points of MR-KNN

Despite its strengths, the MR-KNN algorithm also presents several limitations. The most significant is that the algorithm has **quadratic complexity** with respect to the combined size of the training and test sets. For each

test point, distances must be computed against all training instances in each partition. This means that scaling the test set is just as costly as scaling the training set, and very large test sets can cause memory issues when using vectorised distance computation. This was encountered in this work, where the test set was capped at 5000 points to avoid out-of-memory errors when building the full distance matrix.

The algorithm also **cannot benefit from predicate pushdown or other query optimisations**. Since KNN requires computing distances against all training instances to guarantee finding the true k nearest neighbors, there is no way to filter out training instances before the distance computation. This is a limitation of the algorithm, not an implementation issue.

The **DataFrame implementation introduces serialisation overhead** through `applyInPandas`, which requires converting Spark DataFrames to pandas DataFrames and back for each partition. On a single-node environment, this overhead dominates and results in speedup values below 1.0 relative to the sequential baseline, as shown in Table 2. While this overhead would be amortised on a multi-node cluster, it represents a significant disadvantage compared to the RDD implementation in single-node setups.

Finally, the algorithm **requires the full test set to be accessible to every map task**. In the original Hadoop implementation this is achieved by distributing the test file to all nodes. In our Spark implementation this is handled by passing the test matrix as a closure via `functools.partial`. While this works well for moderate test set sizes, it becomes a bottleneck for very large test sets as the entire test matrix must be serialised and sent to every executor.

5.1.3 • Single-Node vs Multi-Node Performance

A key observation from the experimental analysis is that the scalability benefits of the MR-kNN algorithm could not be fully demonstrated in our experimental setting. Both the RDD and DataFrame implementations were run on a single-node environment.

On a true multi-node cluster, as used in the reference paper, increasing the number of map tasks directly reduces execution time since each map task runs on a separate physical node simultaneously. In contrast, on a single node, increasing the number of map tasks introduces additional Spark scheduling and serialisation overhead without any corresponding reduction in computation time.

This distinction is important when interpreting the speedup results from Experiment 1. The RDD implementation achieves genuine speedup of up to 6x over the sequential baseline even on a single node, because Spark's RDD execution model can pipeline tasks efficiently within a single machine. The DataFrame implementation, however, is consistently slower than its sequential baseline due to the additional `applyInPandas` overhead. On a multi-node cluster, both implementations would be expected to achieve near-linear speedup with increasing number of mappers.

5.2 Comparison with the Paper

The results obtained in this work share several similarities with those reported in the reference paper, but with important differences that arise from the different experimental environments.

In terms of **accuracy**, our results are consistent with the paper. For k=5 on the full training set, we obtain accuracy values of approximately 0.55 for both implementations, which is close to the paper's reported value of 0.5280 for the same configuration. The general trend of increasing accuracy with larger k is also consistent, with k=7 yielding the highest accuracy in both our experiments and the paper.

Regarding the **map and reduce time breakdown**, our results closely mirror the paper's findings. In both cases, the map phase accounts for the vast majority of total execution time while the reduce phase remains negligible. The paper reports reduce times of 1–50 seconds depending on k and number of mappers, whereas our reduce times are between 1–10 seconds. This difference occurs because of our much smaller dataset size.

The most significant **difference** between our results and the paper lies in the speedup analysis. The paper reports near-linear speedup with increasing number of mappers, achieving up to 149x speedup with 256 mappers for k=1. In contrast, our experiments show decreasing speedup with increasing mappers for the RDD implementation, and speedup below 1.0 for the DataFrame implementation. As discussed previously, this discrepancy is entirely attributable to the difference in execution environment. The paper used a 15-node Hadoop cluster whereas our experiments were conducted on a single-node environment.

Overall, the implementation correctly reproduces the algorithmic behaviour described in the paper, and the differences in experimental results are well understood and consistent with the constraints of the available infrastructure.

6 CONCLUSION

In this work, we implemented the MapReduce-based k-Nearest Neighbor algorithm proposed by Maillo et al. (2015) [3] using the Apache Spark API. The implementation follows the map and reduce phases described in the paper. The map phase is implemented in order to achieve distance computation across training partitions, and the reduce phase merges local candidate lists using an O(k) sorted-list merge to recover the exact global k-nearest neighbors. The algorithm was evaluated on the Poker Hand dataset from the UCI Machine Learning Repository across two sets of experiments, and the performance of the RDD-based implementation and DataFrame-based implementation is compared.

The experimental analysis confirmed that both implementations produce equivalent and exact classification accuracy, validating the correctness of the parallelisation approach. The DataFrame implementation demonstrated faster execution times than the RDD implementation at larger training set sizes due to fully vectorised numpy distance computation, while the RDD implementation incurred lower overhead at smaller scales and with higher mapper counts. Both implementations exhibited approximately linear scaling with training set size, consistent with the theoretical complexity of the algorithm.

The speedup analysis highlighted the critical importance of the execution environment for the MR-KNN algorithm. The RDD implementation achieved consistent speedup of up to 6x over the sequential baseline, while the DataFrame implementation was slower than its sequential counterpart due to `applyInPandas` serialisation overhead. Both implementations showed decreasing speedup with increasing number of map tasks, which is a direct consequence of the single-node serverless environment and contrasts with the near-linear speedup reported in the paper on a 15-node cluster. This demonstrates that the MR-KNN algorithm is designed for distributed multi-node infrastructure, and we cannot take its full advantage on a single-node setup.

As future work, running these experiments on a true multi-node cluster would allow for a more complete validation of the algorithm's scalability properties and a fairer comparison with the paper's results. Additionally, addressing the memory constraint on the test set size, for example by processing test points in batches within the map function, would allow the full test set to be used and would bring the experimental setup closer to that of the original paper.

REFERENCES

[1] Xindong Wu and Vipin Kumar, eds. *The Top Ten Algorithms in Data Mining*. Data Mining and Knowledge Discovery. Chapman & Hall/CRC, 2009.

[2] Andrew Frank and Arthur Asuncion. *UCI Machine Learning Repository*. `http://archive.ics.uci.edu/ml`. Accessed: 2026-03-31. 2010.

[3] Jesús Maillo, Isaac Triguero, and Francisco Herrera. "A MapReduce-Based k-Nearest Neighbor Approach for Big Data Classification". In: *Proceedings of the 14th IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2015, pp. 167–172.

APPENDIX

Code

All code implemented for this project can be found here.